Translating SNOMED CT with LLMs

A systematic, reproducible pipeline for medical terminology translation

Korean (KHIS) — Procedures first

Project demo & walkthrough · 2026-06-17

What we'll cover

1. **The core idea** — an LLM as a batch transformer with a template prompt
2. **The task** — SNOMED CT and why translation is hard
3. **The app** — building translations as *flows*; importing data
4. **Using existing translations** as context (the hierarchy trick)
5. **Capturing every variable** for systematic experimentation
6. **The model & speed** — how fast, and what "all of SNOMED" costs
7. **Auto-improving the prompt** with GEPA
8. **Live demo** — driving the app from an AI assistant (MCP)

Part 1 — The core idea

One prompt template, thousands of items

Write the instructions **once**, leave **slots** to fill — then loop over every concept.

The template (written once)

```
SYSTEM
  Follow these rules:
  {style guide}

USER
  Examples:
  {context}
  Translate: {term}
```

Filled in for one term

```
SYSTEM
  Follow these rules:
  …Hangul only; write
  초음파검사 as one word…

USER
  Examples:
  MRI of abdomen
    → 복부 자기공명영상
  Translate: MRI of pelvis
```

→ model returns **골반 자기공명영상**

The actual template we use

`scripts/translation/translate_korean_with_lookup.py`

System (the rules — same for every term):

```
You are a medical terminology translator specialising in English→Korean
translation of SNOMED CT clinical terms... Return ONLY the Korean (한글).

# Korean SNOMED CT translation style guide
{style_guide}
```

User (filled in per term):

```
Here are similar Korean SNOMED translations for reference:
{paired_translations}          ◀── retrieved context (more on this soon)

Translate this SNOMED CT procedure term from English to Korean.
English: {english}             ◀── the term to translate
Korean:
```

Part 2 — The task

SNOMED CT, and why this is hard

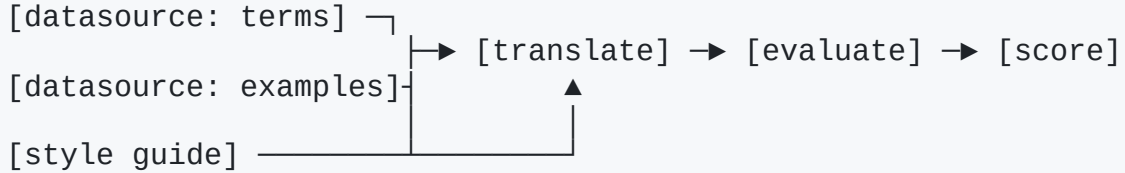
- **SNOMED CT** = the international clinical terminology. ~**350,000** active concepts (procedures, findings, body structures, substances…).
- Each concept has an English name + a place in a **hierarchy** (parents/children).
- Countries maintain **national extensions** with translations — but coverage is partial.
- **Korea (KHIS):** procedures are the 2026 priority; tens of thousands still untranslated.

Translation isn't word-for-word: spacing rules, Sino-Korean vs native terms, laterality, fixed compounds, modality idioms… **specialist conventions matter.**

Part 3 — The app

Translations are built as *flows*

A **flow** is a graph (DAG) of typed building blocks you wire together:



Node	What it does
`datasource`	Load a set of terms / examples
`style_guide`	The rules block (the system prompt)
`translate`	Run the LLM over the terms
`evaluate`	Score against references
`optimize`	**GEPA** — improve the style guide

Built visually in the wizard; stored as JSON in `configs/flows/`.

A real flow (the baseline)

`configs/flows/ko_baseline.json` — abbreviated

```
{ "id": "ko_baseline", "project": "project",
  "nodes": [
    {"id":"terms",     "type":"datasource", "params":{"source":"kr_test_split"}},
    {"id":"exemplars", "type":"datasource", "params":{"source":"pooled_legacy"}},
    {"id":"sg",        "type":"style_guide","params":{"path":"...ko_v5_1.md"}},
    {"id":"translate_full","type":"translate",
       "params":{"model_key":"gemma4-26b"},
       "inputs":{"terms":"terms","exemplars":"exemplars","style_guide":"sg"}},
    {"id":"evaluate_full","type":"evaluate",
       "inputs":{"translations":"translate_full","reference":"terms"}}
  ]}
```

Every arrow in the picture = one `inputs` wire here. Nothing hidden.

Importing data — SNOMED in, examples out

Data sources are registered once, then reused by any flow (`configs/sources/`):

- **SNOMED national extension** (`snomed_national_extension`): point at an RF2 release folder → it extracts concept IDs, English names, and any **existing translations** in the language refset.
- **CSV** (`csv`): a test/eval split with column→role mapping (`sctid`, `en`, `target`).
- **Pooled examples**: all enabled sources merged into a retrieval corpus.

```
{ "id": "kr_snomed", "kind": "snomed_national_extension",
  "rf2_root": "data/korean/SnomedCT_ManagedServiceKR_.../",
  "language_refset_id": "21000267104" }
```

The hierarchy trick — reuse what's already translated

When we translate a term, we give the model **related terms that are already translated** in the official extension — retrieved automatically.

```
To translate:   "MRI of pelvis"   →   ?

A sibling — same scan, different body part — is already in the KR release:

    "MRI of abdomen"   →   복부 자기공명영상      (abdomen · MRI)

The model keeps 자기공명영상 (= MRI) and just swaps the body part:

    "MRI of pelvis"    →   골반 자기공명영상  ✓   (pelvis · MRI)
```

- Retrieval = **BGE-M3** embeddings in a **Qdrant** vector DB (semantic + keyword).
- The model anchors new translations to **established, human-approved** terminology — borrowing the rendering of "MRI" from a sibling and changing only the site.
- Reference-free where no translation exists yet; consistent where it does.

Part 4 — Systematic experimentation

The point: capture *every* variable

Translation quality depends on many knobs. The app makes each one **explicit, named, and versioned** — so results are comparable and reproducible.

Variable	Where it lives
Model + quantization	`configs/models.json` (`model_key`)
Style guide version	`style_guide/*.md` (node input)
Example pool / source	`configs/sources/*`
Retrieval depth (top-N)	node params
Temperature / sampling	node params
Evaluation recipe	project (`evaluation.scorers`)

Change one knob → new flow → new run → directly comparable score. **No spreadsheets, no "what did I run last time?"**

How we measure quality (rigorously)

Three complementary signals — no single number trusted alone:

- **Exact / chrF match** vs the KR reference — strict, fast, automatic.
- **LLM-as-judge** — labels each output `ACCEPTABLE / PARTIAL / WRONG` with reasoning.
- **Back-translation & cross-lingual similarity** (BGE-M3) — *reference-free*, so it works on the long tail where no human translation exists.

Part 5 — The model & speed

Gemma 4 26B-A4B — small where it counts

- **26B total parameters, but only 4B *active* per token** (Mixture-of-Experts).
  → quality of a big model, speed/cost closer to a 4B model.
- **4-bit quantized** (NVFP4) → ~13 GB → runs on a **single workstation GPU**.
- Served locally via **vLLM** (no data leaves the building — matters for health data).

`gemma-4-26B-A4B-it` · `configs/models.json` → `gemma4-26b` / `gemma4-26b-nvfp4-remote`

How fast? Measured throughput

Prior measurement — `docs/gemma4_26b_nvfp4_ablation_2026-04-21.md`, 774-concept batch, concurrency 32

Model	Quant	Throughput	Note
Gemma 4 26B-A4B	**NVFP4**	**~32 / sec**	production default
Gemma 4 26B-A4B	AWQ-4bit	~15 / sec	prior default
Qwen 122B-A10B	GPTQ-Int4	~0.5 / sec	64× slower, no quality gain

~32 concepts per second, on one workstation GPU, fully local.

What would "all of SNOMED" cost?

At ~**32 concepts/sec** (current local Gemma 4 setup):

Scope	Concepts	Time
KR untranslated **procedures**	~55,000	**~29 minutes**
All active SNOMED CT	~350,000	**~3 hours**
Same, on the AWQ build (~15/s)	~350,000	~6.5 hours
Same, on Qwen 122B (~0.5/s)	~350,000	~8 days

> The entire terminology, translated in an afternoon — then the real work (review & refinement) begins, on a complete first draft.

Estimates extrapolate measured throughput; real runs add lookup + I/O overhead.

Part 6 — Auto-improving the prompt

GEPA: let the system rewrite its own rules

The style guide is the highest-leverage knob. **GEPA** improves it automatically.
(**G**enetic **E**volutionary **P**rompt **A**daptation, Pareto-based) — a loop:

```
start from the current style guide
   ▼
①  translate a training sample with it
   ▼
②  score every output, collect the failures
   ▼
③  reflection LLM reads the failures, proposes guide edits
   ▼
④  keep only the best-scoring edits  (Pareto = best trade-offs)
   ↺  the winner becomes the next guide — repeat
```

A model **reads its own mistakes** and rewrites the rules to fix them.

GEPA as a flow

The same building blocks — just add an `optimize` node:

- Inputs: a **train** split (to learn from) and a **dev** split (to validate on).
- Output: a new `style_guide_ko_*.md`, ready to drop into a translation flow.
- Budget presets: `light / medium / heavy` (how much compute the reflection gets).

`scripts/optimization/run_gepa.py` · `pipelines/stages/optimize.py`

What GEPA actually produced (a diff)

Real edits from a v5.1 → v5.4 GEPA run, distilled from scoring failures:

```
  **Spacing (띄어쓰기).**
- Korean terms are predominantly space-separated by word unit.
+ *  Default: space-separated by word unit.
+ *  CRITICAL EXCEPTIONS (fixed compounds, NO spaces):
+      초음파검사  → NEVER 초음파 검사
+      림프조영상  → NEVER 림프 조영상

+ ## Laterality preference
+ | Context              | Preferred  | Alternative |
+ | Internal organs      | 우측 / 좌측 | 오른쪽 / 왼쪽 |
+ | Surface anatomy      | 오른쪽/왼쪽 | 우측 / 좌측  |
```

The system **discovered** these rules from its own errors — the diff is reviewable by a human expert before adoption.

Part 7 — Live demo

Driving the app from an AI assistant (MCP)

MCP lets an AI assistant (Claude) call the app's functions directly — `list_flows`, `duplicate_flow`, `set_node_params`, `run_flow`, `get_run`.

Demo: "Run the translation flow at temperatures 0, 0.5, and 1.0 and compare."

```
for t in [0.0, 0.5, 1.0]:
    duplicate_flow("ko_baseline", new_name=f"temp {t}")
    set_node_params(new_flow, "translate_full", {"temperature": t})
    run_flow(new_flow)            # → job_id
get_run(job_id) ...               # compare headline scores
```

Temperature = how "creative" vs deterministic the model is. 0 = same answer every time.

Why the MCP connector matters

- The app is **not** just a UI — it's a set of operations an agent can orchestrate.
- "Sweep these 5 settings and tell me which wins" becomes **one instruction**, not an afternoon of clicking.
- Same audit trail: every agent-launched run is logged, scored, reproducible.

> The human sets the **question**; the assistant runs the **experiment**;
> the app keeps the **evidence**.

Recap

- LLM + **template prompt** = a batch translator for SNOMED CT.
- **Flows** make each translation experiment a reproducible recipe.
- We reuse **already-translated hierarchy** as context — anchoring to approved terms.
- Every **variable is captured** → systematic, comparable experiments.
- **Gemma 4 26B-A4B**: ~32 concepts/sec → all of SNOMED in ~3 hours, locally.
- **GEPA** rewrites the rules from its own mistakes — human-reviewable diffs.
- **MCP** lets an assistant run the whole workbench by conversation.

Next: full procedures long-tail run · GEPA on NVFP4 baseline · SME review loop

Thank you — questions?

Code: `pipelines/` · Flows: `configs/flows/` · Models: `configs/models.json`

Optimisation: `scripts/optimization/run_gepa.py` · MCP: `mcp_server/server.py`